

LET'S
ROCK

THE
FUTURE

FIAP

GRADUAÇÃO

CURSOS PRESENCIAIS E ON-LINE

Prof. Dr. Marcus Grilo

Prof. Dr. Marcus Grilo

Ciência da Computação

Computer Organization and Architecture

Representação de Dados. Bits, bytes, tipos de dados, caracteres ASCII, representação de informações na memória e comunicação Serial.

Prof. Dr. Marcus Grilo



<https://www.linkedin.com/in/marcus-grilo/>



profmarcus.grilo@fiap.com.br

Representação de Dados

"Quando eu escrevo a palavra **CASA** no computador, como o computador consegue armazenar essa informação?"

"O computador entende a palavra **CASA**?"

CASA → caracteres → códigos → números → bits → bytes

C → 67 → 01000011

A → 65 → 01000001

S → 83 → 01010011

A → 65 → 01000001

Representação de Dados

Portanto, a palavra:

CASA

pode ser representada na memória como:

01000011 01000001 01010011 01000001

C → 67 → 01000011

A → 65 → 01000001

S → 83 → 01010011

A → 65 → 01000001

Conceito fundamental: o bit

Um bit pode assumir apenas dois valores:

0 ou 1

Podemos imaginar o bit como um interruptor:

DESLIGADO → 0 ou LIGADO → 1

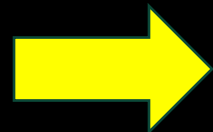
Um conjunto de 8 bits forma um byte:

01001010

Esse byte possui:

8 bits

E pode representar:



00000000 → 0

11111111 → 255

Conceito fundamental: o bit

Portanto, com 8 bits conseguimos representar:

256 valores diferentes

A fórmula é:

$$2^8 = 256$$

O byte em Python

Mostrar um byte:

```
byte = 42

print("Decimal:", byte)
print("Binário:", "{:08b}".format(byte,))
```

Resultado:



```
Decimal: 42
Binário: 00101010
```

1	→	00000001
10	→	00001010
25	→	00011001
50	→	00110010
100	→	01100100
128	→	10000000
255	→	11111111

O byte em Python

Representação de dados e programação

```
idade = 20
altura = 1.75
nome = "Maria"
aprovado = True

print(idade)
print(altura)
print(nome)
print(aprovado)
```



Tipo	Exemplo
int	20
float	1.75
str	"Maria"
bool	True

Para o programador, esses dados possuem tipos diferentes. Entretanto, na memória do computador, tudo precisa ser representado por bits.

"Se o computador trabalha com números, como ele representa uma letra?"

Tabela ASCII.

Por exemplo:

A → 65

B → 66

C → 67

Código Padrão Americano para o Intercâmbio de Informação (do inglês *American Standard Code for Information Interchange - ASCII*, pronunciado [áski]).

Um sistema de representação de letras, algarismos e sinais de pontuação e de controle, através de um sinal codificado em forma de código binário.

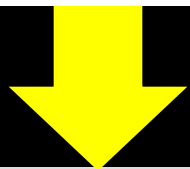
Tabela: ASCII

No Python podemos descobrir o código de um caractere utilizando `ord()`:

```
letra = "A"

codigo = ord(letra)

print("Letra:", letra)
print("Código ASCII:", codigo)
print("Binário:", "{:08b}".format(codigo,))
```



```
Letra: A
Código ASCII: 65
Binário: 01000001
```

Por exemplo:

A → 65
B → 66
C → 67

E depois:

A → 65 → 01000001

65	01000001	41	A
66	01000010	42	B
67	01000011	43	C
68	01000100	44	D
69	01000101	45	E
70	01000110	46	F
71	01000111	47	G

Sequência

"A"
↓
65
↓
01000001
↓
memória

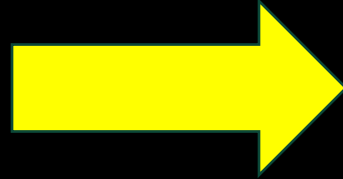
Python: Analisar uma Palavra

```
texto = "CASA"

for letra in texto:
    codigo = ord(letra)

    print(
        letra,
        "->",
        codigo,
        "->",
        "{:08b}".format(codigo,)
    )
```

Resultado:

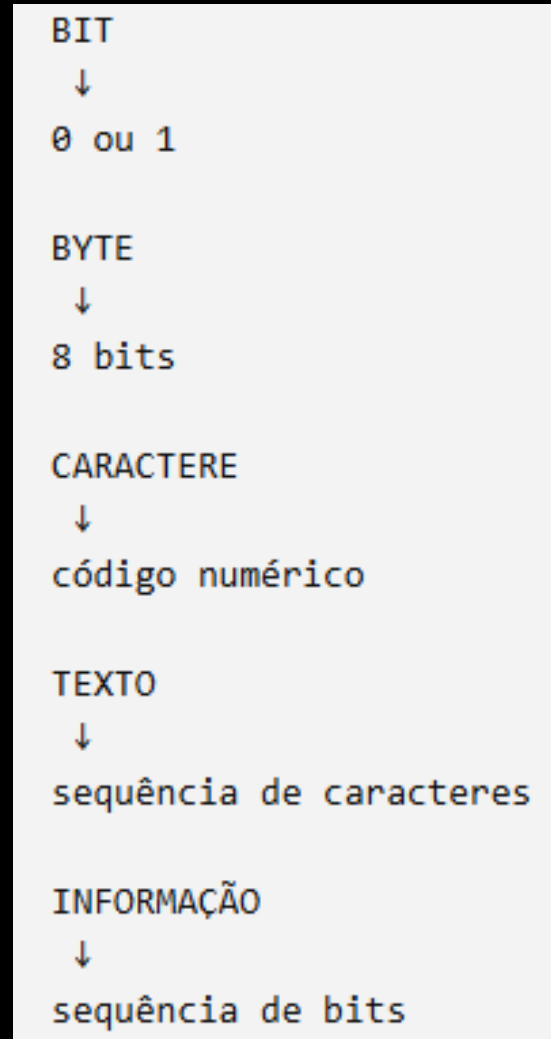
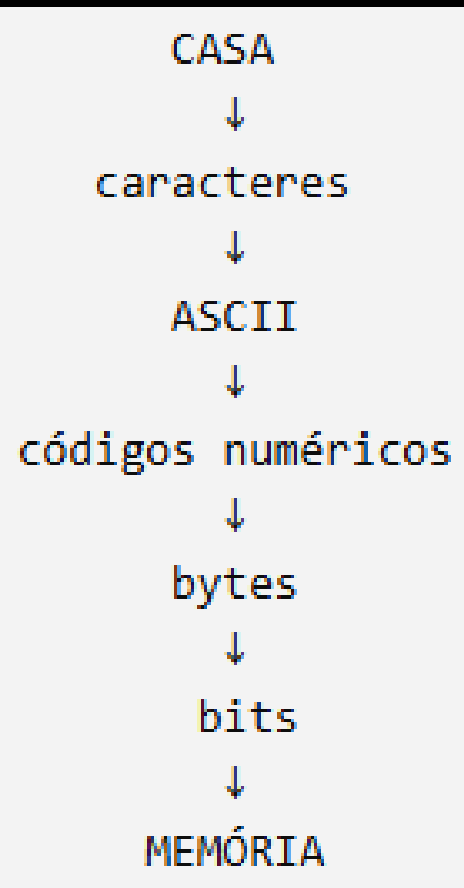


C	->	67	->	01000011
A	->	65	->	01000001
S	->	83	->	01010011
A	->	65	->	01000001

Quanto bytes são necessários para armazenar "CASA", considerando caracteres ASCII?

Resumo

Quando você escreve CASA no computador, o que realmente vai para a memória?



Agora é com você

1 - DECIFRE A MENSAGEM

Você interceptou uma mensagem transmitida por um computador.

A mensagem está representada somente em código binário.

Cada grupo possui 8 bits e representa um caractere ASCII.

Desenvolva um programa em Python capaz de descobrir o que está escrito.

Agora é com você

1 – MENSAGEM

```
01000110 01001001 01000001 01010000 00100000 01100101 00100000
01100001 00100000 01101101 01100101 01101100 01101000 01101111
01110010 00100000 01100110 01100001 01100011 01110101 01101100
01100100 01100001 01100100 01100101 00100000 01100100 01100101
00100000 01110100 01100101 01100011 01101110 01101111 01101100
01101111 01100111 01101001 01100001 00100000 01100100 01101111
00100000 01000010 01110010 01100001 01110011 01101001 01101100
```

Neste exercício, estamos considerando cada caractere ASCII como um byte.

Agora é com você

1 – Dica

Python

```
mensagem = "01001000 01001111 01001100 01000001"

grupos = mensagem.split()

print(grupos)
```

Resultado:

```
['01001000', '01001111', '01001100', '01000001']
```

Python

```
split()
```

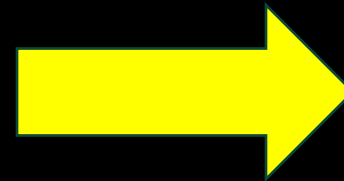
divide uma string em partes.

Agora é com você

2 - Desenvolva um programa que receba uma mensagem e apresente sua representação em bytes e bits.

1. Solicite uma mensagem;
2. Converta cada caractere em seu código;
3. Converta cada código para binário;
4. Mostre os resultados no terminal;
5. Informe a quantidade de caracteres;
6. Informe a quantidade de bytes;
7. Informe a quantidade total de bits.

Resultado:



Exemplo

Digite sua mensagem: HELLO

H -> 72 -> 01001000

E -> 69 -> 01000101

L -> 76 -> 01001100

L -> 76 -> 01001100

O -> 79 -> 01001111

Caracteres: 5

Bytes: 5

Bits: 40